Virtual Memory

Key Topics

- Demand Paging
- Copy on Write
- Page Replacement & Page Replacement Algorithms
- Trashing

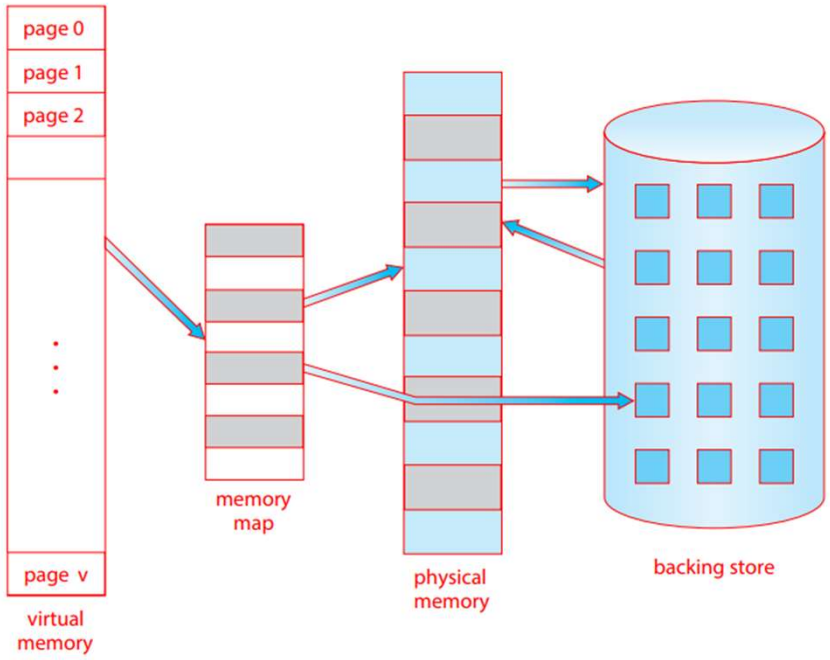
Objectives

- Define **virtual memory** and its benefits
- Illustrate how pages are loaded using **demand paging**
- Explain the process of handling a **page fault**
- Describe the **copy-on-write** optimization
- Apply and analyze various **page-replacement algorithms** (FIFO, Optimal, LRU)
- Understand frame allocation policies and the problem of **thrashing**

Introduction

What is Virtual Memory?

- **Definition:** A technique that separates the **logical memory** (as seen by the process) from the **physical memory** (the actual RAM)
  - This allows a program to have a very large, contiguous address space, even if the physical memory is small or fragmented
  - Only parts of the program that are currently needed are loaded into physical memory
- **Benefits:**
  - **Larger Address Space:** A process can be larger than the physical memory
  - **Increased Multiprogramming:** More processes can be partially loaded into memory, increasing CPU utilization
  - **Simplified Programming:** Programmers no longer need to worry about the physical size of memory
  - **Efficient Sharing:** Allows for shared libraries and memory between processes

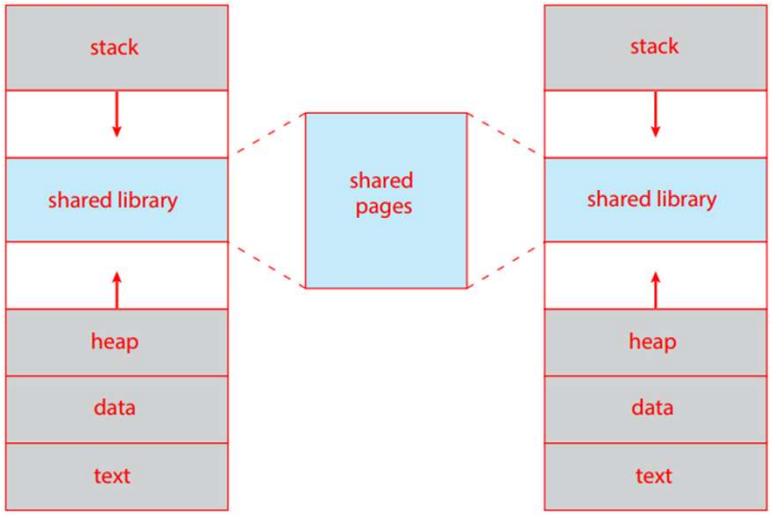


Virtual Address Space Layout

- The OS provides each process with its own virtual address space, typically starting at address 0
- The space between the **heap** (which grows up) and the **stack** (which grows down) is a "sparse" part of the address space
- Virtual memory allows the heap and stack to grow as needed, only allocating physical frames when those pages are actually used

Sharing with Virtual Memory

- Virtual memory facilitates efficient sharing of resources:
- **Shared Libraries (e.g., standard C library):**
  - Libraries can be mapped read-only into the virtual address space of multiple processes
  - All processes share the *same* physical pages of the library, saving memory
- **Shared Memory:**
  - Processes can share a region of memory for communication
  - The shared region is mapped into the virtual address space of each sharing process, but the physical pages are shared
- **Copy-on-Write (fork()):**
  - Pages can be shared during process creation, speeding up the fork() operation

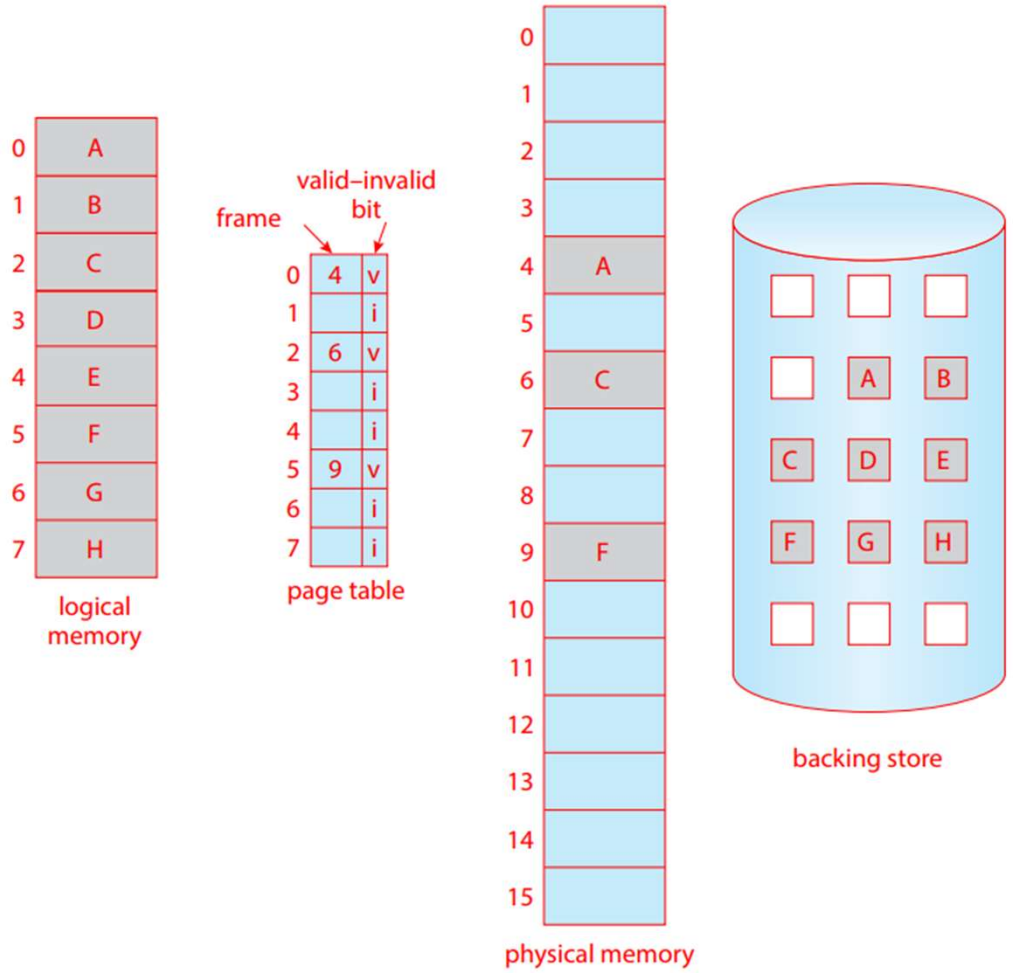


Demand Paging

Demand Paging: Basic Concept

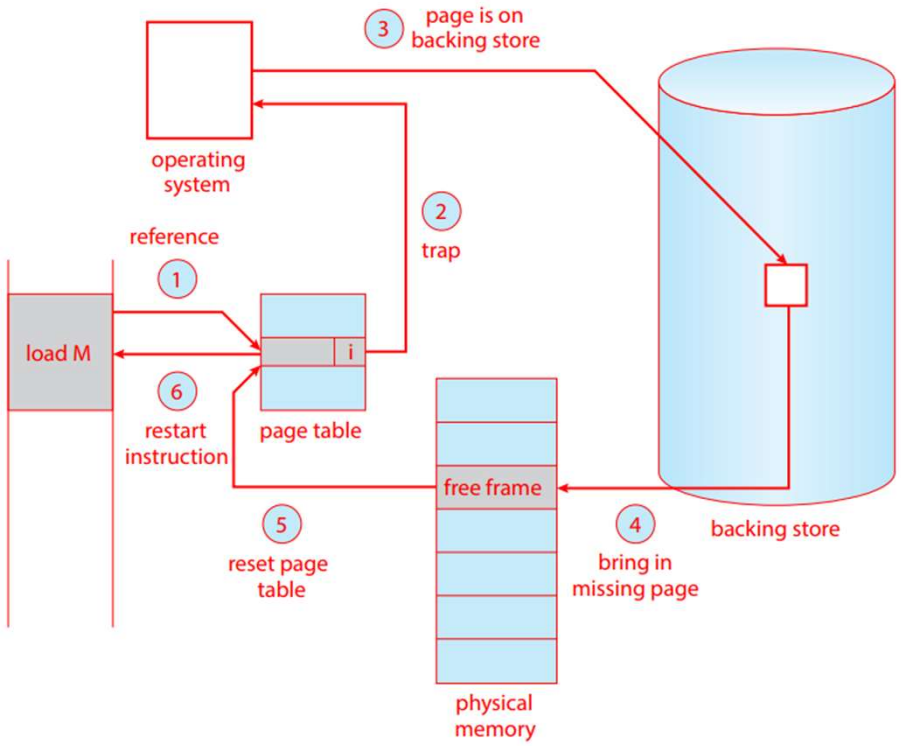
- **Demand Paging** is the strategy of loading pages only when they are needed (demanded) during execution. It's a form of "lazy swapper".
- **How it works:**
  - When a process starts, no pages are loaded into memory
  - The hardware traps to the OS when the process tries to access a page not in memory - this trap is a **page fault**
- **Hardware Support:** We need a **valid-invalid bit** in the page table
  - **V (Valid):** The page is in logical memory *and* physical memory
  - **I (Invalid):** The page is either:
    - Not a valid part of the process's address space
    - A valid page that is currently on the backing store (disk)
- **Benefits:**
  - Less I/O needed to load a process
  - Less memory needed
  - Faster response time
  - More users can be supported

Demand Paging Example



Handling a Page Fault

1. **Trap** to the operating system.
2. **Check:** Was the memory access valid? (i.e., is the page part of the process's logical address space?) If no, terminate the process. If yes, page it in.
3. **Find** a free physical frame from the **free-frame list**. (If no frame is free, a page-replacement algorithm is run).
4. **Schedule** an I/O operation to read the desired page from the backing store into the free frame.
5. **Modify** the page table to set the page's entry to valid and update the frame number.
6. **Restart** the instruction that was interrupted by the page fault.



Demand Paging Performance

- **Locality of Reference:** Programs tend to access pages in "local" clusters, meaning page faults are relatively infrequent once the working set is loaded
- **Effective Access Time (EAT):**
  - ma = memory access time
  - p = probability of a page fault ($0 \leq p \leq 1$)
  - page_fault_time = time to service the page fault
  - **EAT = (1 – p) × ma + p × (page fault time)**
- **Impact:**
  - Page fault service time is dominated by the disk I/O and is very slow (e.g., 8 milliseconds).
  - Memory access time is very fast (e.g., 200 nanoseconds)
  - **A high page-fault rate is disastrous for performance**
  - Example: If p = 0.1% (1 in 1,000), EAT = 8.2 microseconds -> This is a 40x slowdown!
  - We need p to be extremely low for good performance

Copy on Write

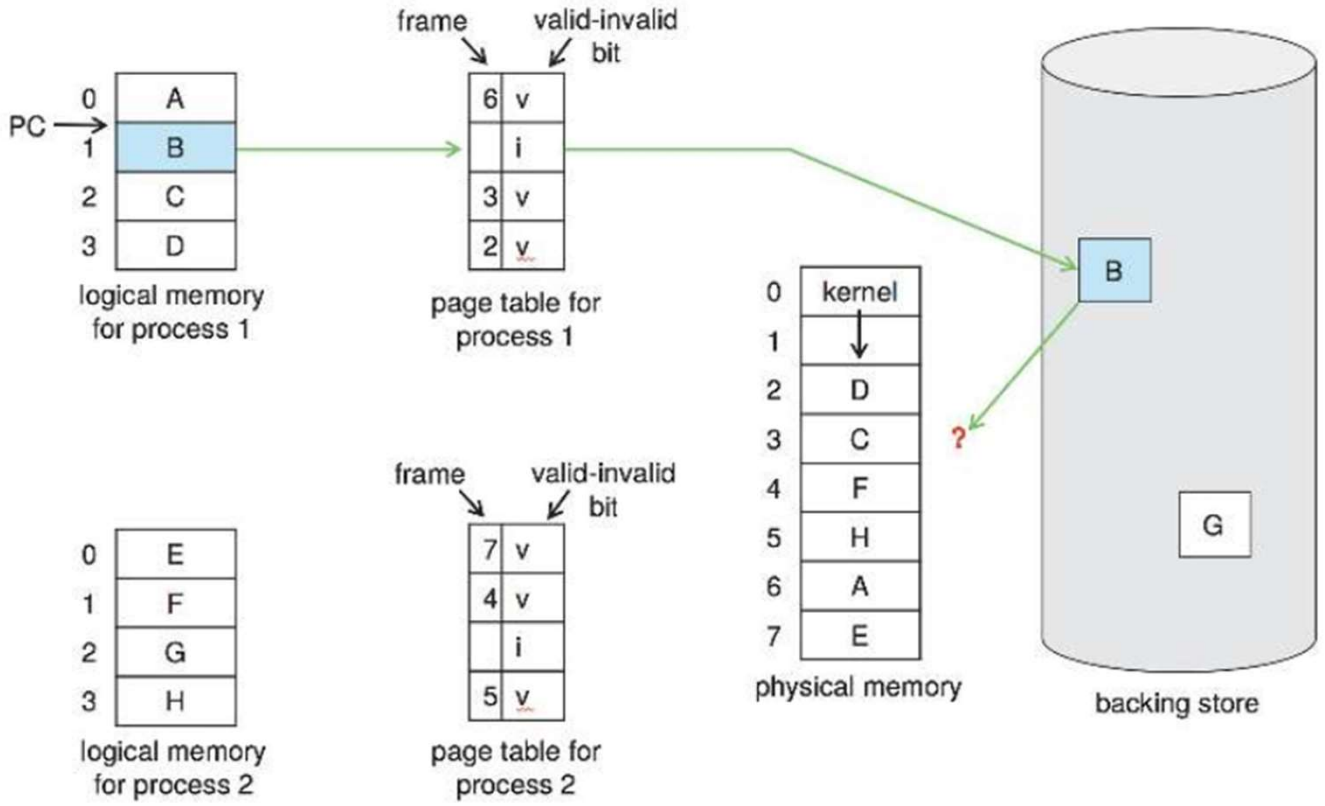
- COW is an optimization for the fork() system call that avoids unnecessary page copying
- **How it works:**
  1. When fork() is called, the parent and child processes are allowed to **share** all pages
  2. The shared pages are marked as **copy-on-write**
  3. If *either* process tries to **write** to a shared page, a trap occurs
  4. The OS creates a **copy** of that specific page, maps it to the writing process's address space, and marks the page as writable
  5. The writing process modifies its private copy; the other process's page is unchanged
- **Benefit:** Only pages that are modified are ever copied. Read-only pages (like code) are shared indefinitely

Page Replacement

Page Replacement

- **Problem:** A page fault occurs, but there are **no free frames** in physical memory
- **Solution:** We must "evict" a page (a "victim") from memory to make room.
  1. Select a victim frame
  2. If the victim page is **"dirty"** (modified), write it to the disk
  3. Load the desired page from disk into the now-free frame
  4. Update the page tables for both the victim and the new page
- **The "Dirty Bit" (Modify Bit):** A hardware bit that is set to 1 by the hardware whenever a page is written to. This saves us from writing "clean" pages back to disk

Page Replacement – No Free Frame



Basic Page Replacement & The Modify Bit

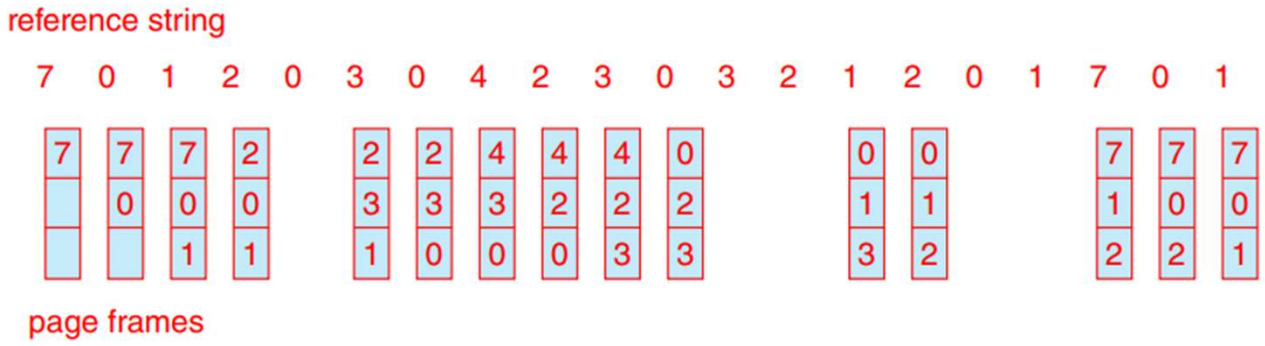
- Page replacement doubles the page-fault service time (one page-out, one page-in)
- **Optimization: The Modify Bit (or Dirty Bit)**
  - A bit associated with each page in the page table
  - Set by hardware whenever any byte in the page is **written** to
- **How it's used:**
  - When a page is selected for replacement, the OS checks its modify bit
  - If bit is **0 (clean):** The page has not been modified. It can just be overwritten, as the copy on disk is still valid
  - If bit is **1 (dirty):** The page *has* been modified. It **must** be written to the backing store before being replaced
  - This significantly reduces I/O time if the page is "clean"

Page Replacement Algorithm: FIFO

- **First-In, First-Out:** Replaces the page that has been in memory the longest
- **Implementation:** Managed with a simple FIFO queue
- **Pro:** Very simple to implement
- **Con: Belady's Anomaly**: This is a rare but serious problem where *increasing* the number of available frames can *increase* the number of page fault

FIFO Example

- **Example (3 frames):**
- Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- Page Frames: 7 7, 0 7, 0, 1 2, 0, 1 (fault, 7 replaced) 2, 0, 1 (hit) 2, 3, 1 (fault, 0 replaced) 2, 3, 0 (fault, 1 replaced) 4, 3, 0 (fault, 2 replaced)
- **Total Page Faults: 15**

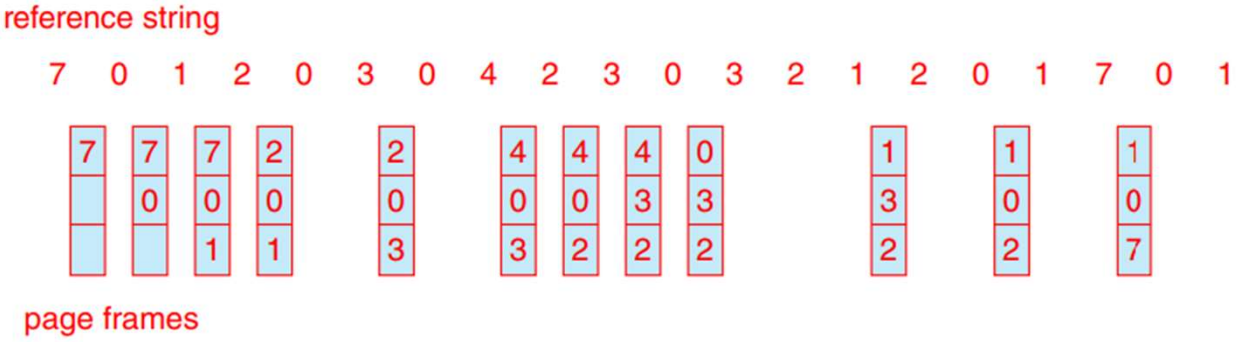


Page Replacement Algorithm: LRU

- **Least Recently Used:** Replaces the page that has **not been used for the longest period of time in the past**
- **Rationale:** Uses the recent past as an approximation of the near future (locality of reference)
- **Pro:** Very good performance, approximates OPT well. Does not suffer from Belady's Anomaly
- **Con:** Hardware implementation is very costly
  - **Counter Method:** Each page entry has a time-of-use register. (High overhead)
  - **Stack Method:** Keep a stack (doubly-linked list) of page numbers. (High overhead

LRU Example

- **Example (3 frames):**
  - Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
  - Page Frames: 7 7, 0 7, 0, 1 2, 0, 1 (fault, 7 replaced; 7 is LRU) 2, 0, 1 (hit) 2, 0, 3 (fault, 1 replaced; 1 is LRU) 2, 0, 3 (hit)
  - 4, 0, 3 (fault, 2 replaced; 2 is LRU)
- **Total Page Faults: 12**

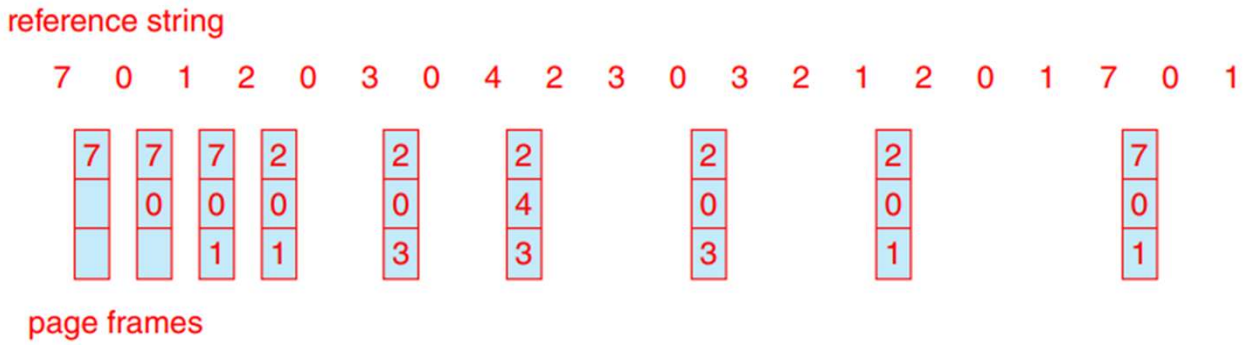


Page Replacement Algorithm: Optimal (OPT)

- **Policy:** Replace the page that will **not be used for the longest period of time in the future**
- **Pro:** Guarantees the lowest possible page-fault rate
  - Does not suffer from Belady's Anomaly
- **Con:** Impossible to implement, as it requires knowledge of the future
- **Use:** Serves as a theoretical benchmark to compare other algorithms against

OPT Example

- **Example (3 frames):**
  - Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
  - Page Frames: 7 7, 0 7, 0, 1
  - 2, 0, 1 (fault, 7 replaced; 7 is not used until ref 18) 2, 0, 1 (hit) 2, 0, 3 (fault, 1 replaced; 1 is not used until ref 14)
- **Total Page Faults: 9**



Frame Allocation

Allocation of Frames: Minimum

- How many frames should we allocate to each process?
- **Minimum Number of Frames:**
  - There is a minimum number of frames that *must* be allocated.
  - This is defined by the **computer architecture**
  - Why? An instruction may reference multiple pages
    - For example, an instruction that moves data from one memory location to another (e.g., MVC) could straddle a page boundary *and* reference two operands that *also* straddle page boundaries
  - We must allocate enough frames to hold all pages a single instruction can reference

Allocation of Frames: Algorithms

- Given m frames and n processes:
  - **Equal Allocation:**
    - Split the frames equally: m/n frames for each process
    - Simple, but doesn't account for different process sizes
  - **Proportional Allocation:**
    - Allocate available memory to each process according to its size (s_i)
    - Total size $S = \Sigma s_i$
    - Allocation for process p_i is $a_i \approx (s_i / S) \times m$
    - This is a more fair approach based on process needs
- *Both policies can be modified by process priority*

Allocation of Frames: Global vs. Local

- **Local Replacement:**
  - If a process needs a new page, it must select a victim from **its own** set of allocated frames
  - The number of frames for this process is constant
  - Performance depends only on the process's own paging behavior
- **Global Replacement:**
  - If a process needs a new page, it can select a victim from the set of **all** frames
  - One process can "steal" a frame from another
  - A process's frame count can change over time
  - **Pro:** Generally results in greater system throughput and is more common
  - **Con:** A process's execution time can vary, as it's affected by the paging behavior of other processes

Trashing

Trashing

- **Problem:** A process does not have enough frames to hold its current **locality of reference** (the set of pages it is actively using)
- **Symptom:** The process quickly page-faults, replaces a page, and then immediately faults for the page it just replaced
- **Definition: Thrashing**
  - A process is **thrashing** if it is spending more time paging than executing
- **System-level effect:**
  - CPU utilization drops.
  - The OS (CPU scheduler) sees the low utilization and increases the degree of multiprogramming, admitting a new process
  - The new process steals frames, causing other processes to thrash
  - CPU utilization drops even further -> **System throughput plunges**

Trashing: Working-Set Model

- Prevents thrashing by ensuring a process has its required locality in memory.
- **Working-Set Window (Δ):** A parameter specifying a fixed number of recent page references (e.g., the last 10,000).
- **Working Set (WSSi):** The set of unique pages in the most recent Δ references for process p_i. This is an approximation of the process's locality.
- **System State:**
  - D = Σ WSSi (Total demand for frames)
  - m = Total physical frames
  - If **D > m**, thrashing will occur.
- **Policy:** The OS monitors the working set of each process. If D > m, the OS must suspend a process to free up frames.

Trashing: Page-Fault Frequency

- A more direct approach: control the page-fault rate
- **Policy:**
  - Establish an "acceptable" page-fault rate (upper and lower bounds)
  - If **PFF > Upper Bound**: The process needs more frames. Allocate another one
  - If **PFF < Lower Bound**: The process has too many frames. Remove one from its set
- If the PFF is high and no free frames are available, a process must be suspended (swapped out)

Virtual Memory